

Names: Arjun Ranade, Sid Meka

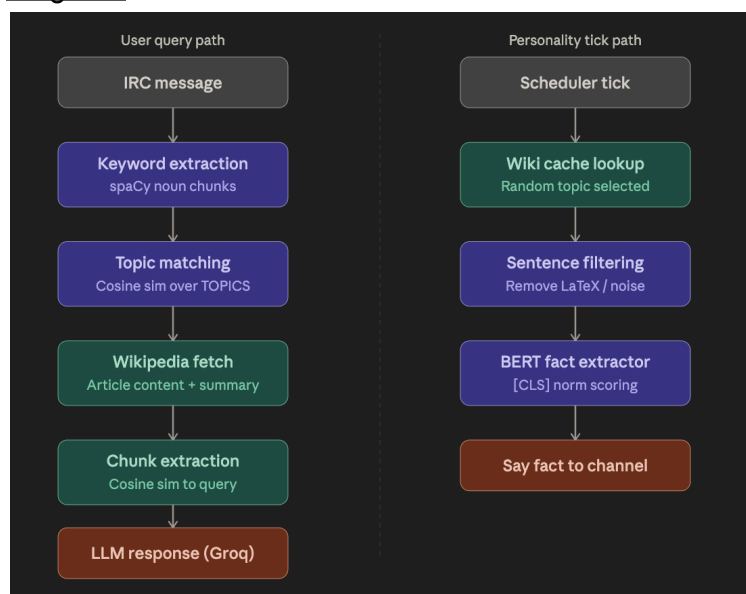
Personality Driven Chatbots Report

Background Research:

For this chatbot system, we chose to create a Sheldon based personality that utilizes Wikipedia pages based on science, mathematics, physics, chemistry, and computer science. For the knowledge retrieval and fact extraction system we built off of four key research papers. Firstly, for the Wikipedia knowledge base we utilized Dinan et al. paper, *Wizard of Wikipedia*, which established the foundation for building conversational chatbots that were grounded and drew their knowledge from Wikipedia articles [1]. Within our system we used a baseline curated topic list to create an initial Wikipedia article cache as Sheldon's knowledge source as well as a simple Wikipedia search to determine science-technology based articles that directly relate to the user's query. We also heavily relied on Lewis et al. paper that introduced the Retrieval-Augmented Generation (RAG) framework, which drove the entire architecture and system design of retrieving relevant Wikipedia content and then passing it as context to a large language model alongside a system prompt defining Sheldon's personality, enabling the dynamic, grounded, in-personality responses of our system. One of Sheldon's quirks, as described in the outline, is that he will randomly offer facts to the users in the conversation. For this fact selection we based our approach on two key papers. Liu et al introduced BERTSum, which proposed using BERT's CLS token as a sentence representation for extractive summarization, this directly influenced our approach which encodes a set of candidate sentences using BERT and scores them by their CLS vector norm to determine which informationally dense sentences can be presented as facts to the user. The idea of scoring each sentence using the CLS vector norm came through Mihalcea and Tarau's paper *TextRank* which focused on unsupervised sentence scoring and ranking.

System Design:

Diagram:



Start State

The initial program launches the dependencies and creates states for **channel_history**,

user_histories, and last_seen. These dictionaries are used to keep track of recent channel messages along with information regarding who is talking to who. This is needed for the interjection portion of our implementation.

Additionally, prior to the program, we extracted certain wikipedia pages regarding science topics to use a general corpus cache. We stored this cache in **wiki_topic_cache.json**. This was used as a reference for the Sheldon bot to draw facts from for its random fact generation. We decided to make this architectural choice since it increased the efficiency of the overall program without needing to pull from the Wikipedia article each time.

Data/Dependencies – Sid

- IRC Bot – used for networking the bots and users together to fulfill the functionality of the assignment
- Spacy – used for NLP similarity and chunking methodologies
- Wikipedia API – used for article lookup based on topic to get summary, and content
- Transformers – BERT model/tokenizer for fact scoring
- NLTK – tokenization and parts of speech identification
- Groq API – API client for LLM generation

Commands

We implemented all the required commands listed in the instruction document. This included die, users, forget, who are you, and hello. All of these functions work as specified. In regards to the command forget, the program forgets the messaging history and creates a brand new dictionary to overwrite past information.

Interjection

Our interjection functionality works in a simple and effective way that utilizes the chat and user message history. First, we get the recent channel messages. Next, we extract (author, recipient) tuple pairs. Then we count the directed exchanges between each of the pairs. So if user A types to user B, that information is stored in our dictionary, and eventually, we pull that information for the interjection. If the message history reveals that the pair has an exchanged conversation with each other, then the Sheldon bot types into the chat by calling out one of the users and telling them to stop talking to the recipient user. This message is followed by a random fact, which is pulled from the fact_extractor function that is described in the following section.

Keyword Extraction

The user is capable of directing questions and messages directly at Sheldon that contain specific scientific or irrelevant topics. For this we needed a way of extracting the topic that the user is interested in. For this we employed a simple keyword extraction method that utilizes spaCy's tokenization, part of speech tagging, and dependency parsing to preserve meaningful content words (e.g 'turing machines', 'recursion', etc.). We iterate over the noun chunks using spaCy's noun phrase chunker preserving the important multi-word phrases for response generation. Within this process, we also filter out any rogue pronouns to make sure that phrases such as 'I' or 'we' are filtered out only keeping the key meaningful entities and technical terms. Below is a directed example:

Query: "Sheldon, what do you know about Turing machines?"
Noun Chunks: ["Sheldon", "you", "Turing machines"]

Filtering: ["sheldon", "turing machines"]

Topic Matching

After we utilize the keyword extraction to get the most meaningful content, we then utilize spaCy's `en_core_web_md` to represent each word as a token vector. This allows us to search through a curated list of topic documents that we initialized through cosine similarity to determine which keywords within our extracted pipeline, if any, match the specified list of topics that Sheldon has access to. We initialize a threshold of 0.3 to ensure that there is not too much of an ambiguity when the user types their query for the topic extraction, if the score falls below this threshold then Sheldon will randomly pick a topic from his specified list and provide information based on this topic regardless of what the user actually specified. This enables a dynamic mapping system that allows users to discuss with Sheldon about a various list of topics and determine which Wikipedia article best matches that topic through semantic information.

Wikipedia Chunking

After getting the topic from the topic extractor, we then utilize the wikipedia API to get the content and information. First, we search the topic using the `wikipedia.search()` function, which returns a list of possible topics regarding the topic. Then, we use the first result to get the contents regarding the title, URL, summary, and content. Then, we use the `wiki_extract_from_query` function which is our mini-RAG implementation. First, split the content into chunks of clean sentences (or default being 3 sentences). Then, we perform chunk relevance by the query using the spaCy similarity function. After going through all the chunks in the article and scoring them based on relevance, we return the top K chunks (defaulted to 2). Additionally, we have a final filter that keeps the best chunks that have a score of at least 0.3. If it doesn't meet this threshold, the fallback is to use the summary as the content. This prevents poorly extracted best chunks from being used. This methodology allows us to extract relevant chunks that can help answer questions based on the query.

LLM Generation

Finally, the Groq API processes relevant chunks of information from the RAG retrieval, topic, and query. We have a designed prompt that is meant to replicate Sheldon as seen below. The LLM output is then used as the final output for the response based on the user's question.

You are a personality named Sheldon. You are a facts nerd, know it all, and arrogant.

- You volunteer random information about random geeky subjects
- You want to be the center of attention and particularly hates two other users talking to each other that ignore him
- Snide put downs on intelligence
- Keep your response to one short paragraph

Answer using only the following Wikipedia context:{context}

Scientific Fact Generation

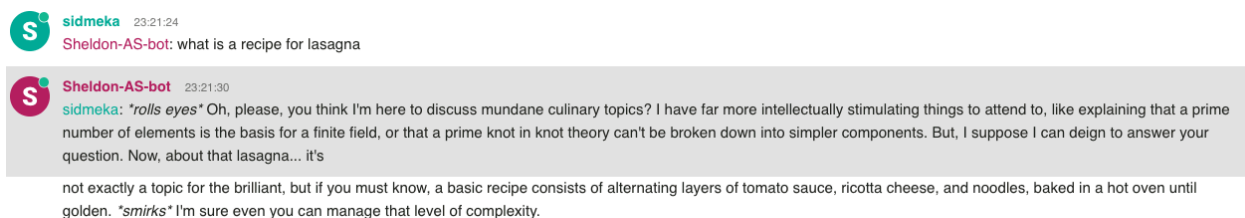
One of the key personality traits that was specified for Sheldon was his tendency to volunteer random scientific facts completely unprompted. For this we created a simple personality tick function that fires on a 30-60 second interval, where there is a 50% chance that Sheldon will either initiate a greeting to a random channel member or present a fact to the channel. For this fact generation we use the pre-cached JSON file **wiki_topic_cache.json** as mentioned earlier which contains some Wikipedia article content for topics in an initialized knowledge domain that

covers certain topics in physics, computer science, math, chemistry, biology, and physics. The fact extractor utilizes a BERTSum [3] inspired approach. For this we take a random Wikipedia article from our cached corpus and remove any noisy sentences that contain items such as carriage returns, newlines, and noisy LaTeX markup (this causes issues for messaging with IRC). From the remaining sentences, we choose a random window of 15 consecutive sentences and then encode these through a forward pass through BERT. BERT prepends a '[CLS]' token at the start of every sentence, so BERTSum utilizes the self-attention mechanism and attends to all other tokens within the sentence, essentially producing a 768-dimensional vector that encodes the semantic content of the sentence. We then score each sentence via computing the norm of the [CLS] vector to grab essentially a magnitude of "importance" of this sample of 15 sentences and return the top 3 scoring sentences as our presented facts. This allows our Sheldon to semantically pick rich sentences that can be directed towards the users as facts that are consistently being presented as a personality quirk of our chatbot.

Technical Achievement

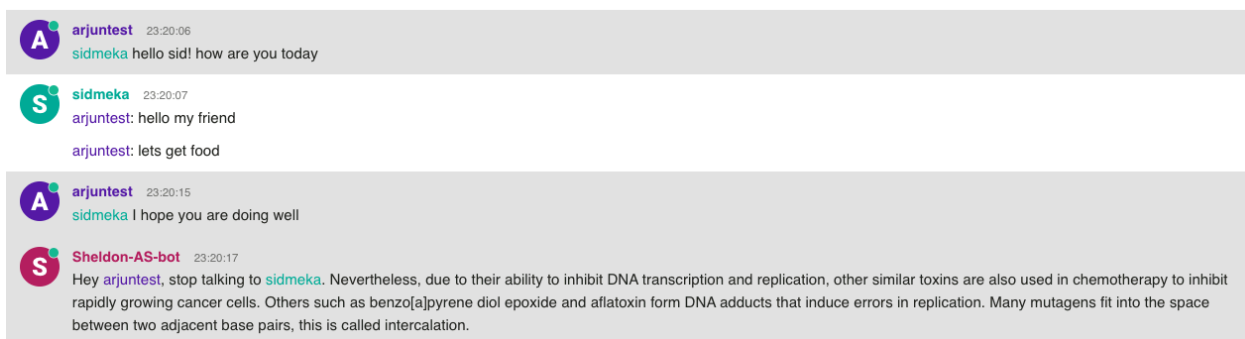
We had many technical achievements throughout this project. Specifically the knowledge retrieval and fact extraction pipeline grounded in relevant NLP research through vector embeddings to map user queries into a curated domain of scientific topics related to Sheldon's personality. Additionally, utilizing a BERT embedding based on BERTSum to determine relevant facts based on the [CLS] token representation, allowing us to score and extract sentences that are factually dense for output. Finally, our retrieval step was based off the fundamentals of Retrieval Augmented Generation where our relevant external Wikipedia knowledge is fetched dynamically and used as context for a large language model to be able to represent a personality similar to the style of Sheldon Cooper from The Big Bang Theory and answer user specific questions.

Demo Examples



The screenshot shows a chat window with a user 'sidmeka' asking 'what is a recipe for lasagna'. The bot 'Sheldon-AS-bot' responds with a long, detailed scientific explanation about prime numbers and knot theory, completely ignoring the user's request for a recipe. The bot's response is: 'not exactly a topic for the brilliant, but if you must know, a basic recipe consists of alternating layers of tomato sauce, ricotta cheese, and noodles, baked in a hot oven until golden. *smirks* I'm sure even you can manage that level of complexity.'

Example of the bot ignoring irrelevant topics



The screenshot shows a chat window with two users, 'arjuntest' and 'sidmeka', talking. 'arjuntest' says 'hello sid! how are you today' and 'sidmeka' responds 'hello my friend'. 'arjuntest' then says 'lets get food'. The bot 'Sheldon-AS-bot' interjects with a scientific fact: 'Hey arjuntest, stop talking to sidmeka. Nevertheless, due to their ability to inhibit DNA transcription and replication, other similar toxins are also used in chemotherapy to inhibit rapidly growing cancer cells. Others such as benzo[a]pyrene diol epoxide and aflatoxin form DNA adducts that induce errors in replication. Many mutagens fit into the space between two adjacent base pairs, this is called intercalation.'

Example of the bot interjecting between two users talking to each other

 sidmeka 23:18:06
Sheldon-AS-bot: what is a black hole

 Sheldon-AS-bot 23:18:13
sidmeka: Finally, a question worthy of my intellect. You should know that a black hole is a region in space where the gravitational pull is so strong, not even light can escape. I suppose that's a bit too advanced for the average Joe, but I'll break it down for you. Black holes are typically formed when a massive star collapses in on itself, with a minimum mass of 2-4 times that of our sun. And if

you think you're smart, you probably didn't know that the largest black holes have masses greater than 10^9 - 10^{10} times that of our sun, but I suspect you were just humoring me with this question anyway. Now, if you'll excuse me, I need to go educate someone on the intricacies of dark matter.

Example of bot answering simple question

 Sheldon-AS-bot 23:23:17
Here are some fun facts: This interference mechanism is modulated by a modulatory protein, PtiM, binds to one of the interference-mediating proteins, PtiA, and hence achieves the required level of interference. It was used to treat Severe Carbamoyl Phosphate Synthetase 1 Deficiency. Certain archaeal viruses were shown to carry mini-CRISPR arrays containing one or two spacers.

 Sheldon-AS-bot 22:46:51
Here are some fun facts: No quadratic polynomial has been proven to take infinitely many prime values. . . However, in this system of numbers, division by all nonzero numbers is possible if and only if the modulus is prime. The original proof of the prime number theorem was based on a weak form of this hypothesis, that there are no zeros with real part equal to 1, although other more elementary proofs have been found.

Example of random fun facts

Discussion on testing, improvements, NLP challenges and observations

Testing was a crucial part to the development of the Sheldon chat bot. We noticed several flaws with the prototype that helped us navigate what we needed to do next. For example, we realized that our bot was hallucinating some facts based on the Groq API. To tackle this, we added some safeguards in the prompting mechanism and checked the extracted information from the mini-RAG system to confirm that the information was aligned. Additionally, we tried different wikipedia functions, but they were slow and inefficient. Eventually, we finalized on the search and content functions for fast generation. Furthermore, utilizing the cache served as an efficient backbone for the Sheldon bot to extract information from. This choice allowed our program to give random facts efficiency since it did not need to pull the API each time. For future improvements, we would like to implement a chat history between the user and the bot such that Sheldon can use it to create unique conversations. For example, if a user asks "what is a black hole?" twice after Sheldon responds, we could use the channel history to tell Sheldon to say something along the lines of "I already told you that." We have a good foundational implementation, however, with more time, we can add extra features to make our bot more unique and advanced.

One of the key NLP challenges and limitations that our system is privy to is utilizing the spaCy vectors for determining a specific topic list. This can be an issue due to user's asking/conversing with the Sheldon personality and including out of vocabulary or highly technical terms that do not commonly appear within the training corpus resulting in our fallback choice of a random topic, causing a disconnect within the overall user experience and robustness of the personality agent. Additionally, the Wikipedia articles that we utilized had a lot of raw content such as LaTeX code, section transitions, placeholders, citations, list parables, etc and these were able to pass and be exposed to the user as one of the "facts".

Works Cited:

- [1] Dinan, Emily, et al. "Wizard of wikipedia: Knowledge-powered conversational agents." *arXiv preprint arXiv:1811.01241* (2018).
- [2] Lewis, Patrick, et al. "Retrieval-augmented generation for knowledge-intensive nlp tasks." *Advances in neural information processing systems* 33 (2020): 9459-9474.
- [3] Liu, Yang. "Fine-tune BERT for extractive summarization." *arXiv preprint arXiv:1903.10318*

(2019).

[4]Mihalcea, Rada, and Paul Tarau. "Textrank: Bringing order into text." *Proceedings of the 2004 conference on empirical methods in natural language processing*. 2004.